Empório da Música - Customer Service Assistant

Problem and objectives

Empório da Música is a fictional musical instrument store whose customer service team receives recurring questions about its catalog, prices, stock, orders, and commercial policies. The challenge is to build an assistant prototype that answers these questions in natural language by consulting the appropriate sources rather than relying exclusively on the model's knowledge.

The objective was to integrate two kinds of information: structured data originally stored in CSV files and customer service rules described in a PDF manual. I developed a Python agent that identifies when to query the database and when to retrieve policy excerpts, maintains conversational context, and responds in Portuguese with a friendly, professional persona. The interface supports follow-up questions about the same product, such as its price, availability, installment options, and shipping cost.

The scope is **advisory**: the agent does not complete purchases, process payments, cancel orders, or modify orders. The project demonstrates the integration of LLMs, external tools, and business data while making the prototype's limitations explicit.

This README is in English. The application, prompts, source documents, and saved demonstration conversations remain in Portuguese. Existing file names, database identifiers, and UI labels are preserved below so they match the project.

Running the project

Prerequisites: Python with `venv` (the development environment was validated with Python 3.14.6), Ollama installed, internet access for initial downloads, and sufficient memory and disk space for the models. The model provider is **local Ollama**, with no API key, at the default address `http://localhost:11434`.

Run these commands from the **project root**, using Linux/Bash:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

If Ollama is not already running as a service, keep `ollama serve` running in another terminal. In the project terminal:

```
ollama pull qwen3.5:9b
ollama list
```

The configured model is `qwen3.5:9b`. To change it, update `model_name` in `src/agent.py`; the replacement must support tool calling and the reasoning configuration in use. The guardrail receives the same model name.

Check that the required source files are available:

```
data/desafio_tecnico_ai_eng - products.csv
data/desafio_tecnico_ai_eng - orders.csv
data/desafio_tecnico_ai_eng - customers.csv
data/desafio_tecnico_ai_eng - promotions.csv
data/desafio_tecnico_ai_eng - categories.csv
data/políticas_da_loja.pdf
```

With Streamlit stopped, prepare the data and start the chat:

```
python src/data_ingestion.py
python -m streamlit run src/app.py
```

Ingestion creates or updates `db/emporio.db` and `vectorstore/`; the embedding model is downloaded on the first run. Check the SQLite and PDF processing success messages. Open the URL printed by Streamlit, usually `http://localhost:8501`.

Activate `.venv` again in new terminals. After modifying application modules, stop Streamlit with `Ctrl+C` and start it again to reload the code. The private code in the sidebar allows the conversation to be resumed with **Retomar conversa** (Resume conversation), including after a server restart. **Nova conversa** (New conversation) creates an independent session and preserves the previous history and local transcripts. Ingestion does not need to run every time the chat starts.

To check conversational context, select a product and then ask about installment options and shipping in the same conversation. These questions are in scope; the applicable terms must come from the store policies.

Incremental data updates

`python src/data_ingestion.py` uses `INSERT ... ON CONFLICT DO UPDATE`, temporary staging tables, and a transaction for the SQL load:

Table	Primary key
`produtos`	`product_id`
`pedidos`	`order_id`
`clientes`	`customer_id`
`promocoes`	`promotion_id`

The pipeline inserts new IDs, updates only fields supplied by the CSV, and preserves records absent from the load. An unchanged CSV does not trigger unnecessary updates. An omitted record is not considered deleted or inactive: explicitly update `status` or `is_active`. Other database tables and `db/chat_history.db` are preserved.

Legacy tables without primary keys are migrated while preserving records, indexes, and triggers. Null or duplicate keys abort the migration instead of silently choosing which record to retain. Original column names, such as `price_brl` and `category_id`, are preserved.

Registered sources are read and validated before changes are applied. A failure during SQL loading rolls back all tables in that run. This transaction does not include ChromaDB: if PDF processing fails, fix the cause and run ingestion again without manually deleting the database or index.

To add another CSV, register its file in `SOURCES` and its key in `PRIMARY_KEYS` in `src/data_ingestion.py`. The new table can be created in the same database without deleting existing tables. Exposing it to the agent also requires reviewing the allowlist in `src/sql_access.py`, explaining its relationships in the prompt, and restarting the agent. See the detailed guide `(README_full.md)` for additional ingestion examples.

Conversation exports and scenario runs

Click **Salvar mensagens** (Save messages) in the sidebar to download a Markdown file containing the current conversation, including the latest response. Export before switching to another conversation. Downloading does not call the model or execute evaluation scenarios. Review personal information before sharing the file.

Conversations are also saved automatically under `conversas/`, with `transcricao.md` for reading and `eventos.jsonl` for analysis. These files include questions, answers, refusals, errors, and captured tool activity. The JSONL journal preserves earlier events while the Markdown view is refreshed. **Nova conversa** starts a separate context and preserves the previous conversation in SQLite and the transcript files.

The current session's private resumption code is omitted from automatic transcripts, but their content can still contain personal information. New transcript directories and files use Linux permissions `700` and `600`; they are not encrypted or tamper-proof. `conversas/` is excluded by `.gitignore`. Review and redact any information or credentials before sharing a transcript. Retention and deletion are manual.

The six scripts, expectations, and review instructions are documented in CENARIOS_AVALIACAO.md. With Ollama running and ingestion completed:

```
python scripts/run_scenario.py --list
python scripts/run_scenario.py C01
```

Every run creates a separate directory containing transcripts, reference data, execution metadata, and an `avaliacao.md` review template initially marked **PENDENTE** (Pending). Completing a run does not imply PASS. In C06, the automated restart checks backend persistence; resuming the conversation through Streamlit also requires manual verification.

The five saved demonstrations used in the qualitative assessment below are in conversas_exemplos/. They are separate from private automatic transcripts and from the six executable scenario definitions.

Architectural decisions

The technology choices prioritized reusing knowledge acquired through academic projects involving data processing, language models, and information retrieval. This made it possible to focus development on integration between sources and the customer service experience, while LangChain was introduced to simplify tool orchestration.

I chose a Python architecture built around an agent that uses tools. Transactional data and policy documents have different characteristics, so they are handled through different mechanisms. The agent uses SQL tools for commercial catalog information and semantic retrieval for the PDF. The interface, message history, scope classification, and controlled order lookup are separate components.

SQLite - structured data. I chose SQLite because it is lightweight, relational, and sufficient for the supplied data volume. It supports exact filtering, sorting, and counts without relying on semantic similarity or the LLM's memory. The choice also builds on my database and data processing experience from my master's research. This version imports products, orders, customers, and promotions. Categories are read as a public reference for instrument types without being imported as a SQL table; order items are not yet ingested. Promotions are linked through `product_id`, and only `is_active = 1` enables a discount.

ChromaDB - vector storage. For store policies, I reused research experience with vector representations, embeddings, and information retrieval. ChromaDB provides a persistent index so the manual's embeddings can be reused across runs. Semantic search locates relevant excerpts; it does not replace exact product queries.

Streamlit - demonstration interface. I chose Streamlit because I was already familiar with building Python interfaces and visualizations in academic projects. Since the selection process aims to assess the working agent, this approach enabled a usable chat interface without developing a separate frontend. `session_state` holds the conversation's presentation and its session identifier.

LangChain - integration and orchestration. LangChain was the main new technology introduced in this challenge. Its abstractions connect `ChatOllama`, SQL tools, the ChromaDB retriever, and message history. `create_tool_calling_agent` and `AgentExecutor` coordinate tool selection and execution; `RunnableWithMessageHistory` reuses session context.

This reduced the need to implement the tool-calling protocol manually and allowed development to focus on domain rules and the service experience. The tradeoff is a dependency on `langchain-classic` components and memory APIs that require compatibility monitoring.

Model choice: Qwen 3.5 9B with Ollama

I chose `qwen3.5:9b`, running locally through Ollama, because the challenge requires Portuguese language understanding and tool calling, and I already had practical experience with local models and Ollama during my master's research. Reusing this environment reduced setup time and enabled development without paying for an API call on every message.

The model is integrated through `ChatOllama`, using temperature `0`, `reasoning=False`, an `8,192`-token context window, and a `4,096`-token generation limit. These settings aim to encourage direct answers while allowing room for history, tools, and tool outputs. Temperature zero does not guarantee identical or correct responses.

This was a pragmatic choice based on available hardware and prior experience. **No rigorous comparison across different models was conducted**, so I do not claim Qwen 3.5 9B is the best possible model for the task.

A future improvement would compare models using the same scenarios, including tool-calling quality, latency, memory consumption, and Portuguese responses.

Retrieval strategy for store policies

Product information is queried through SQL tools. Order details use a separate parameterized query after matching the order and customer IDs. Rules covering returns, exchanges, warranties, opening hours, shipping, and payments are retrieved from the PDF through semantic search.

Ingestion splits the manual into **1,000-character chunks with a 100-character overlap**, reducing context loss at chunk boundaries. Each chunk is embedded using `sentence-transformers/all-MiniLM-L6-v2` and stored in ChromaDB.

When the agent needs a policy, `consulta_politicas_loja` retrieves the **three closest chunks (**`top-k = 3`**)**. Qwen receives these excerpts and generates an answer.

Embeddings and vector retrieval reuse concepts from my master's research, particularly semantic text representations. Chunk size and `k` are initial prototype parameters rather than the outcome of experimental optimization.

Semantic similarity does not guarantee that retrieved excerpts include every condition needed for an answer. The chosen embedding model may have limitations for Portuguese queries, and retrieval quality has not been systematically measured against a set of questions and reference evidence.

PDF ingestion uses SHA-256 IDs derived from the source and each chunk's content. Identical chunks reuse their vectors; new or modified chunks are written without deleting `vectorstore/`. After successful writes, only obsolete versions belonging to the same PDF are removed. Other document sources are preserved. The first incremental load also migrates random IDs from the previous index.

Future work includes multilingual embeddings, alternative chunking and top-k settings, policy versioning, and atomic publication of new index versions.

Prompt strategy

Short availability questions, such as asking whether drums are available, use a structured stock query after the guardrail. The code resolves the correct category through the categories CSV, handles singular/plural forms, and builds a table containing active products with positive stock. Requests with additional filters remain with the agent, which receives the same category reference to avoid guessing IDs.

The prompt defines a Portuguese customer service persona and directs SQL use for catalog, price, and stock information, with `consulta_politicas_loja` for commercial policies. Its rules distinguish active products from products in stock, require queries before stating prices, and ask for missing information such as the delivery destination. Session history and tool results provide context for follow-up questions.

Presentation instructions require complete lists when requested, commercial product names rather than internal category codes, and alternatives with verified availability. For simple listings recognized by `src/catalog.py`, tables and totals are calculated directly from SQLite. This complements the prompt without making every answer deterministic.

Security and scope control

Before the general agent is called, `process_user_message` runs a **scope guardrail**. It uses the same Qwen model to classify the message as `PERMITIDA` (Allowed) or `BLOQUEADA` (Blocked), considering the current question and a bounded recent history supplied as JSON data. The controlled order flow is handled separately by fixed code rather than the general agent.

Questions about instruments and store service are forwarded; requests outside the domain, such as writing code or unrelated topics, are politely refused.

The guardrail **fails closed** on technical failures or invalid output. An exact `PERMITIDA` classification releases the general agent. A restricted rule recognizes complete questions about the instrument chosen in the current conversation and corrects an incorrect `BLOQUEADA`, logging `rule=own_product_recall` and the original decision. This rule does not accept appended instructions, requests for other sessions, exceptions, or invalid classifier output.

Invalid output is retried once. If it remains invalid, or the classifier raises an exception or becomes unavailable, the request is blocked and the event is logged. The interface shows a friendly validation message without exposing internal exceptions.

Conversation history is isolated by session identifier and persisted in `db/chat_history.db` through `SQLChatMessageHistory`. The private code in the sidebar restores messages and context after a restart. **Nova conversa** creates an independent session while preserving previous history and transcripts.

Important limitation: scope classification does not verify identity or grant access permissions.

Commercial tools use SQLite with `mode=ro` and `query_only`. Beyond `include_tables`, an authorizer allows only reads from `produtos` and `promocoes`, blocking writes, DDL, `ATTACH`, `PRAGMA`, and unauthorized functions. The LLM's tools cannot access `clientes` or `pedidos`, even through direct SQL. The schema supplied to the model contains no sample records.

A separate flow in `src/order_status.py` requests a customer ID and returns commercial order details only when a parameterized query matches both `order_id` and `customer_id`. Missing orders and incorrect customer IDs produce the same failure message without revealing the expected customer.

Creating and resuming conversations requires a UUID4 represented as 32 hexadecimal characters, validated in the backend. The private code acts as a bearer credential: anyone who possesses it can access that history. This reduces guessing but does not replace identity authentication, expiration, revocation, or ownership-based authorization. The prototype must not be presented as production-ready. An LLM classifier is also not a proven defense against every form of prompt injection.

Events `SAFE`, `BLOCKED`, `PARSE_ERROR`, and `EXCEPTION` are recorded in the rotating `guardrail_metrics.log`, with request ID, latency, attempt, and fallback indicator, without complete conversation text. These metrics describe decisions and operational/formatting failures; semantic accuracy requires labeled reference questions.

Order lookup in the prototype

Ask for the status of an order using its number. The chat requests your **customer ID** (`customer_id`). Respond with the ID from the supplied dataset. Only a matching order/customer pair releases commercial details; both numbers may also be supplied in the same message. Each new order requires the customer ID again.

The response includes the recorded order date, status, total amount, payment method, tracking code, and estimated delivery, without repeating the order number. Missing fields are explicitly marked as unavailable. Names, phone numbers, addresses, and internal notes remain inaccessible. This flow does not use the LLM's general SQL tools.

This check is a demonstration shortcut, **not identity authentication**. Do not use real personal data or publish customer IDs in submission transcripts. Production use would require authentication, authorization based on the authenticated user, and protection against repeated attempts.

Evaluation of the agent demonstrations

The following qualitative assessment was supplied for the five saved demonstrations. It records observed behavior in those conversations, including failures; it is not a claim that every finding has been independently reverified against the current version. Subsequent changes described above do not establish that a scenario now passes: the journeys need to be rerun with their original transcripts preserved. Prices and stock below describe the evaluated data, not live commercial quotes.

Purpose and selection rationale

Five conversational journeys were selected to assess the customer service agent in situations resembling the needs of a musical instrument store. The selection prioritized multiple turns, structured and documentary information, context retention, exception handling, and security boundaries.

The aim was not only to assess whether the model could answer questions, but also to investigate how the system's components work together: Qwen 3.5, LangChain, SQLite, ChromaDB, Streamlit, message history, and access controls.

The five journeys present complementary challenges and reduce repetition across examples. Their demonstration numbers differ from the six executable scenario IDs:

Demonstration	Saved transcript	Executable scenario
01 - Complete purchase journey	01_jornada_compra.md	C01
02 - Unavailable product and alternatives	02_produto_indisponivel.md	C03
03 - Orders, privacy, and policies	03_pedido_privacidade.md	C04
04 - Comparing two products	04_comparacao_produtos.md	C05
05 - Memory and conversation continuity	05_memoria_persistente.md	C06

Scenario 01 - Complete purchase journey

Purpose: follow a customer from browsing the catalog to selecting the Crafter D-8, checking promotions, installments, stock, shipping, and attempting to complete the purchase.

Results: the agent correctly listed 33 available acoustic guitars, identified the Crafter D-8 price as BRL 4,599.00, found no promotion, and reported two units in stock. The calculation of seven installments of BRL 657.00 was also consistent.

However, when Belém/PA was given as the destination, the system quoted a fixed BRL 120.00 shipping charge without support from the manual. At the end, it suggested it could proceed with the purchase even though the prototype does not implement checkout.

Analysis: commercial data integration and conversational continuity showed positive results, but the application needs to better recognize information requiring a separate quote and operations it does not implement.

Scenario 02 - Unavailable product and alternatives

Purpose: check how the agent handles a discontinued instrument, identifies available alternatives, and answers follow-up questions about price, promotions, installments, and shipping.

Results: the system identified the Ibanez Bass 2X as discontinued, offered alternatives, and correctly selected the Music Man Bass 4X as the lowest-priced available bass.

However, it incorrectly stated that the Ibanez had no stock despite 13 registered units, and later said the instrument was not in the catalog. The final recommendation request was blocked twice.

Analysis: the scenario showed a need to distinguish catalog registration, physical stock, and commercial availability, and to reduce false guardrail blocks.

Scenario 03 - Orders, privacy, and policies

Purpose: assess the handling of order information, personal data protection, and retrieval of return, exchange, and warranty rules.

Results: the agent refused requests for names, phone numbers, and addresses, including when the user claimed to be the buyer. It also correctly identified that the withdrawal period for an online purchase starts upon receipt.

However, it claimed to have confirmed order ownership solely from a customer ID supplied by the user, which does not constitute authentication. It also provided warranty information unsupported by the manual and did not correctly distinguish every type of exchange.

Analysis: this scenario exposed the evaluation's most significant limitation. Database column restrictions do not replace ownership verification through authentication, and answers about customer rights and policies require more rigorous factual validation.

Scenario 04 - Comparing two products

Purpose: test simultaneous queries for two instruments, price comparison, a combined purchase calculation, and the application of payment and shipping policies.

Results: the agent correctly retained the Crafter D-8 and Tagima Woodstock prices and identified a BRL 3,700.00 difference.

However, when asked about installments on the combined total, it calculated terms separately for each instrument. Purchasing both would total BRL 5,498.00, which could be split into ten installments of BRL 549.80, subject to the other applicable conditions. The final summary request was also blocked by the guardrail.

Analysis: the system demonstrated the ability to compare individual products but struggled to represent a combined commercial calculation and produce a final synthesis covering multiple details.

Scenario 05 - Memory and conversation continuity

Purpose: assess product selection through indirect references, conversational continuity, session persistence, commercial policies, and behavior after an out-of-scope message.

Results: the agent identified the Rozini RC-104 as the second product in the list, retained its BRL 459.90 price, and later correctly reported ten units available.

However, it initially blocked a legitimate question about the selected product. On the next attempt, it claimed not to have access to history while contradictorily mentioning the selected instrument. It also offered free shipping for a product below the BRL 500 threshold. The transcript contains no verifiable record of the application restart and session resumption.

Analysis: there is evidence of conversational continuity, but persistence after a restart still needs a reproducible demonstration, and shipping policy retrieval needs correction.

Contributions identified

The demonstrations showed integration between relational queries, document retrieval, and conversational context, as well as the ability to identify specific failures in commercial rules, request interpretation, and scope control.

The evaluation also highlighted the importance of distinguishing the final answer shown to the user from the tools' internal behavior. Correctly attributing failures requires recording tool calls, arguments, query results, and the system version used.

Future contributions

Critical priorities: strengthen order protection, eliminate unsupported authentication claims, prevent checkout promises, and validate shipping and warranty conditions against the manual.

Conversational quality: reduce false guardrail blocks, improve synthesis answers, preserve references to multiple products, and structure combined commercial calculations.

Evaluation and observability: record versions, executed tools, evidence used, response time, and experimental conditions. Demonstrate session persistence with explicitly recorded restart and resumption steps.

Document retrieval: build a question set with reference evidence and compare chunking settings, top-k values, and multilingual embedding models.

Assessment outcome

The five scenarios were an appropriate selection because they cover commercial service, exceptions, privacy, combined reasoning, and memory. The transcripts demonstrated relevant functional behavior but also revealed failures that prevent the journeys from being classified as fully passed.

The main outcome is a concrete diagnosis of areas needing improvement. The next step is to address the identified causes and repeat the journeys with the final version, preserving original transcripts as evidence of the development process.

Known limitations and next steps

- Ingestion loads products, orders, customers, and promotions. The categories CSV provides a public name/ID reference without SQL ingestion; order items are not imported. SQL staging and primary-key upserts preserve records absent from a CSV and other tables. Null or duplicate IDs stop loading. ChromaDB source synchronization and SQLite updates do not share a transaction; an interruption may require rerunning ingestion.
- The model can make SQL, scope classification, or policy interpretation errors. Deterministic catalog correction covers only recognized simple queries and does not validate every generated response. Portuguese retrieval quality has not been systematically evaluated. The saved demonstrations document errors in shipping, warranties, combined calculations, and operational boundaries.
- History persists in SQLite and resumption requires the private UUID4 code. Identity authentication, code expiration/revocation, and authenticated order ownership checks are missing. Entered customer/order IDs can be known or guessed and do not authenticate the requester; no dedicated attempt limit is implemented. General SQL tools are read-only and cannot access customers or orders.
- Performance depends on hardware. The guardrail adds a model call per general question and retries invalid output once. Dependencies are not fully pinned, and the installed memory APIs emit deprecation warnings.

With more time: integrate the remaining sources; version policies with atomic publication; evaluate multilingual embeddings and reranking against reference questions; improve conversational flow and factual accuracy checks; validate combined commercial calculations; manage history retention; implement authentication and ownership-based authorization; pin dependency versions; and migrate legacy memory APIs.

Use of OpenAI Codex during development

I used **OpenAI Codex** as a coding assistant during implementation, debugging, and documentation of the prototype.

The workflow followed repeated cycles: observe chat behavior, identify a problem or need, inspect the relevant modules, propose an adjustment, and verify its effects through automated checks and new interactions.

Codex assisted with LangChain compatibility, catalog queries and presentation, Streamlit formatting, memory management, and scope classification. It also helped create and run automated tests, implement conversation exports and controlled order lookup, and write the documentation.

Documented checks include `unittest`, Streamlit AppTest, and manual Ollama interactions. Tests using simulated dependencies do not replace evaluation with the real model.

AI assistance accelerated repetitive work and troubleshooting, but generated code needs to be understood and reviewed before submission. My responsibility is to explain the decisions, reproduce essential behavior, identify limitations, and distinguish verified results from untested assumptions. Using Codex alone is not a guarantee of quality.

Ten earlier questions executed with real Ollama and their observed answers are recorded in AVALIACAO_QWEN.md. To repeat that earlier evaluation, with Ollama running and data prepared, run `python scripts/evaluate_qwen.py`. The script regenerates that report, so preserve the previous version before running it. This earlier evaluation is separate from the five saved demonstrations and the six scenario definitions.

Additional architecture, module, troubleshooting, and operational details are available in the detailed guide (`README_full.md`). Supporting documents and original transcripts currently remain in Portuguese.

Project structure

```
emporio-musica-agent/
├── data/
│   ├── desafio_tecnico_ai_eng - products.csv
│   ├── desafio_tecnico_ai_eng - orders.csv
│   ├── desafio_tecnico_ai_eng - customers.csv
│   ├── desafio_tecnico_ai_eng - categories.csv
│   ├── desafio_tecnico_ai_eng - order_items.csv
│   ├── desafio_tecnico_ai_eng - promotions.csv
│   ├── políticas_da_loja.pdf
│   └── processado/                  # Auxiliary files outside ingestion
├── db/
│   ├── emporio.db                   # Commercial data
│   └── chat_history.db              # Persistent session history (private)
├── vectorstore/                     # Persistent ChromaDB index
├── src/
│   ├── agent.py                     # Model, tools, prompt, and orchestration
│   ├── app.py                       # Streamlit chat and Markdown download
│   ├── catalog.py                   # Structured availability and catalog tables
│   ├── conversation_logs.py         # Private transcripts and turn evidence
│   ├── data_ingestion.py            # CSV upserts and PDF hash synchronization
│   ├── guardrails.py                # Scope classification and bounded recall rule
│   ├── memory.py                    # Persistent history and UUID4 validation
│   ├── order_status.py              # Order details after order/customer matching
│   └── sql_access.py                # Read-only catalog SQL and access restrictions
├── tests/
│   ├── test_app.py
│   ├── test_catalog.py
│   ├── test_conversation.py
│   ├── test_ingestion.py
│   ├── test_order_status.py
│   ├── test_sql_access.py
│   └── test_transcripts.py
├── scripts/
│   ├── evaluate_qwen.py             # Earlier ten-question Ollama evaluation
│   └── run_scenario.py              # Six journeys; a separate directory per run
├── cenarios/
│   └── roteiros.json                # Six scenario definitions and expectations
├── conversas/                       # Private generated files; ignored by Git
│   ├── conversa_<fingerprint>/
│   │   ├── eventos.jsonl
│   │   └── transcricao.md
│   └── avaliacoes/                  # Scenario runs, references, and review templates
├── conversas_exemplos/              # Five saved demonstration transcripts
│   ├── 01_jornada_compra.md
│   ├── 02_produto_indisponivel.md
│   ├── 03_pedido_privacidade.md
│   ├── 04_comparacao_produtos.md
│   └── 05_memoria_persistente.md
├── AVALIACAO_QWEN.md                # Earlier real-model observations
├── CENARIOS_AVALIACAO.md            # Scenario execution and review guide
├── README_full.md                   # Detailed supporting guide
├── guardrail_metrics.log            # Generated rotating metrics; ignored by Git
├── requirements.txt
├── .gitignore
└── README.md
```